

UJIAN TENGAH SEMESTER (UTS) – TAHUN AKADEMIK 2025/2026

PROGRAM STUDI SISTEM INFORMASI (S1)

Mata Kuliah : Sistem Enterprise (Enterprise Systems)

Tipe Soal : Esai Analitis & Studi Kasus

Dosen Pengampu : Irmayansyah, S.Kom.,M.Kom

Semester/Kelas : IV (Empat)

Sifat Ujian : Tutup Buku

Metode : Luring

Nama : Muhammad Akmal Syarif

Nim:241572010004

Soal 1:

Jelaskan mengapa fenomena Information Silos dianggap sebagai "penyakit mematikan" bagi organisasi yang sedang berkembang menjadi sebuah Enterprise. Hubungkan jawaban Anda dengan risiko biaya operasional dan akurasi pengambilan keputusan oleh manajemen puncak

Jawaban:

Kenapa "Silo Informasi" Bisa Membunuh Bisnis yang Sedang Berkembang?

Di dunia korporat, *information silos* atau silo informasi itu sering banget jadi "penyakit kronis". Begitu sebuah perusahaan makin besar dan bertransformasi ke skala *enterprise*, sekat-sekat informasi ini bakal langsung merusak skalabilitas bisnis. Efek dominonya fatal: muncul ego sektoral di mana tiap divisi sibuk dengan urusannya masing-masing, dan perusahaan kehilangan kelincahan (*agility*) untuk merespons pergerakan pasar.

Secara garis besar, ada dua dampak mengerikan yang akan langsung menggerogoti perusahaan:

1. Anggaran Bocor & Operasional yang Inefisien

Ketika data tersumbat dan tidak mengalir antar-divisi, jangan heran kalau biaya operasional tiba-tiba membengkak. Masalah yang biasanya muncul antara lain:

- **Duplikasi Kerja & Sistem:** Divisi A dan Divisi B tanpa sadar mengerjakan hal yang sama atau membeli *software* yang fungsinya mirip karena tidak ada komunikasi. Ini jelas membuang-buang anggaran.

- **Buang-buang Waktu:** Tim menghabiskan terlalu banyak energi hanya untuk melakukan sinkronisasi dan rekonsiliasi data secara manual yang melelahkan dan rawan *human error*.
- **Rantai Pasok Berantakan:** Contoh paling klasik adalah ketika tim *sales* dan tim produksi tidak sinkron. Hasilnya? Prediksi stok meleset total—bisa jadi barang menumpuk di gudang, atau justru kehilangan momentum karena kekurangan barang saat permintaan pasar sedang tinggi.

2. Manajemen Puncak Seperti "Disetir" dalam Kondisi Buta

Bagaimana mungkin jajaran direksi bisa mengambil keputusan strategis yang tepat jika perusahaan tidak memiliki *Single Source of Truth* (satu sumber data acuan yang valid)?

Tanpa adanya transparansi data, pihak manajemen hanya bisa melihat data yang sepotong-sepotong atau bahkan data jadul yang sudah basi. Imbasnya, keputusan penting sering kali terlambat diambil, atau yang lebih parah, salah sasaran karena pemimpin terjebak dalam *blind spots* (titik buta).

Kesimpulannya: Silo informasi adalah musuh dalam selimut bagi perusahaan. Di level internal, ia menyumbat efisiensi yang bikin keuangan bocor. Sementara di level eksekutif, ia membuat jajaran direksi "buta" dalam menentukan arah dan strategi masa depan perusahaan.

Soal 2 :

Dalam siklus hidup BPM, mengapa tahap Process Discovery (As-Is) dan Process Analysis tidak boleh dilewati begitu saja sebelum melakukan implementasi software ERP? Apa konsekuensi teknis dan bisnis jika perusahaan langsung melakukan tahap Redesign (To-Be) tanpa memahami kondisi awal?

Jawaban:

Langsung Bikin Sistem ERP Baru Tanpa Evaluasi? Ibarat Operasi Tanpa Diagnosis Dokter

Melewatkan tahap *Process Discovery* (As-Is) dan *Process Analysis* demi bisa langsung lompat ke tahapan *Redesign* (To-Be) itu ibarat naik meja operasi tanpa diagnosis dokter. Hasilnya sudah pasti nekat dan berbahaya. Banyak perusahaan gagal total saat implementasi ERP hanya karena mereka terburu-buru memperbaiki sistem, tanpa benar-benar paham apa akar masalah dan realitas yang terjadi di lapangan.

Jika kedua tahapan awal ini diabaikan, siap-siap saja menghadapi dua hantaman besar berikut:

1. Kekacauan dari Sisi Bisnis & Operasional

- **Penolakan Massal dari Karyawan:** Tim di lapangan bakal protes atau menolak mentah-mentah sistem baru karena dinilai tidak realistis dan justru mempersulit kerjaan mereka.
- **Operasional Lumpuh Total:** Risiko kemacetan operasional sangat tinggi saat ERP mulai diaktifkan (*Go-Live*). Bisnis bisa mendadak macet karena sistem baru tidak siap menghadapi realitas kerja.
- **Anggaran Bocor Amat Parah:** Biaya akan membengkak (*budget overrun*) karena Anda harus terus-menerus melakukan revisi desain sistem di tengah jalan akibat banyak hal yang meleset.

2. "Benang Kusut" di Sisi Teknis

- **Migrasi Sampah Data:** Tanpa pembersihan dan analisis di awal, Anda hanya akan memindahkan "sampah data" (*bad data*) dari sistem lama ke sistem yang baru.
- **Kustomisasi yang Kebablasan:** Karena sistem standar tidak cocok dengan proses asli perusahaan, Anda terpaksa melakukan kustomisasi berlebihan (*over-customization*). Efeknya? ERP jadi sangat rapuh, kaku, dan biayanya selangit setiap kali mau di-*update*.
- **Sistem yang "Apatis":** Muncul celah integrasi (*integration gaps*) yang membuat ERP baru kesulitan untuk terhubung atau "ngobrol" dengan aplikasi pendukung lainnya.

Kesimpulannya: Tahap pemetaan proses saat ini (*As-Is*) dan analisis mendalam adalah jangkar realitas perusahaan. Tanpa keduanya, implementasi ERP yang mahal itu cuma akan mendigitalisasi inefisiensi. Alih-alih rapi, Kamu hanya memindahkan kekacauan lama ke dalam sistem digital yang baru.

Soal 3 :

Dalam teori Value Chain, terdapat aktivitas utama (Primary) dan aktivitas pendukung (Support). Argumenkan bagaimana implementasi sistem ERP mampu memperlebar Margin keuntungan perusahaan melalui integrasi kedua aktivitas tersebut! Berikan satu contoh konkret pada aktivitas Inbound Logistics

Jawaban:

Cara ERP Mendongkrak Margin Keuntungan Lewat Integrasi Rantai Nilai

Pada dasarnya, ERP berfungsi sebagai jembatan yang meruntuhkan tembok pembatas antar-departemen. Dengan menghubungkan **Aktivitas Utama (Primary)** dan **Aktivitas Pendukung (Support)**, perusahaan bisa memangkas biaya operasional sekaligus mendongkrak nilai jual produk di mata konsumen. Hasil akhirnya? Margin keuntungan perusahaan otomatis melebar karena efisiensi yang tercipta.

Bagaimana Sinkronisasi Ini Bekerja?

ERP memastikan semua data mengalir secara *real-time* tanpa ada yang tersumbat. Sebagai contoh:

- **Procurement (Support) + Operations (Primary):** Pembelian bahan baku menjadi sangat presisi karena didasarkan pada data kebutuhan riil di lini produksi, bukan sekadar tebak-tebakan.
- **Finance (Support) + Sales (Primary):** Proses persetujuan kredit atau plafon utang untuk pelanggan bisa selesai dalam hitungan detik, sehingga tim *sales* bisa menutup transaksi jauh lebih cepat.

Studi Kasus Nyata: Transformasi di Sisi *Inbound Logistics*

Perubahan drastis akan sangat terasa jika kita melihat perbandingan proses pengelolaan barang masuk (*inbound logistics*) sebelum dan sesudah menggunakan ERP:

- **Sebelum Ada ERP (Gaya Lama):** Proses pengecekan stok di gudang, pembuatan dokumen pesanan ke vendor, hingga urusan bongkar muat barang semuanya dilakukan secara manual. Prosesnya lambat, melelahkan, dan rawan memicu *human error* yang bikin lini produksi mandek akibat bahan baku telat datang.
- **Setelah Menggunakan ERP (Gaya Baru):** Begitu stok barang di gudang menyusut ke batas minimum, sistem akan otomatis menerbitkan *Purchase Order* (PO) dan mengirimkannya ke vendor. Begitu barang pesanan tiba, petugas gudang cukup memindai *barcode*. Dalam satu klik, jumlah stok langsung ter-update, *invoice* keuangan otomatis tercatat, dan bahan baku bisa langsung dikirim ke lini produksi tanpa hambatan.

Kesimpulannya: Integrasi yang dihadirkan oleh sistem ERP berhasil memotong rantai birokrasi dan biaya operasional yang tidak perlu—seperti biaya sewa gudang berlebih akibat penumpukan barang dan biaya administrasi manual. Di saat yang sama, risiko kelangkaan bahan baku bisa ditekan habis. Struktur biaya yang makin ramping inilah yang secara otomatis langsung memperlebar margin keuntungan bersih perusahaan.

Soal 4:

Jelaskan arsitektur Three-Tier (Presentation, Application, Data) pada sistem enterprise. Mengapa pemisahan ketiga lapis ini sangat krusial bagi perusahaan yang membutuhkan aspek Scalability (skalabilitas) dan Security (keamanan) data?

Jawaban :

Mengapa Arsitektur Three-Tier Jadi Standar Wajib buat Sistem Kelas Enterprise?

Dalam membangun sistem berskala besar (*enterprise*), arsitektur *Three-Tier* sudah menjadi kiblat standar yang wajib diikuti. Konsep dasarnya adalah membagi sistem ke dalam tiga lapisan independen yang punya tugas spesifik: *Presentation Tier* (tampilan UI), *Application Tier* (otak atau logika bisnis), dan *Data Tier* (tempat penyimpanan *database*).

Pemisahan ini bukan sekadar biar kelihatan rapi, tapi punya dampak krusial dalam memberikan dua keunggulan utama bagi perusahaan:

1. Skalabilitas yang Jauh Lebih Efisien (*Independent Scaling*)

Kelebihan utama dari sistem ini adalah tiap lapisan bisa ditingkatkan kapasitasnya masing-masing secara mandiri.

- **Contoh Nyata:** Kalau situs atau aplikasi perusahaan tiba-tiba kebanjiran pengunjung, Anda cukup menambah atau memperkuat kapasitas server di *Presentation Tier* (lapisan tampilan) saja.
- **Keuntungannya:** Anda tidak perlu repot-repot menduplikasi sistem atau membayar biaya lisensi tambahan yang mahal untuk lapisan aplikasi maupun *database*. Strategi ini jelas jauh lebih hemat anggaran.

2. Keamanan Data Berlapis Sebagai Benteng Pertahanan

Arsitektur ini secara otomatis menciptakan sistem proteksi jarak jauh yang sangat solid bagi aset perusahaan.

- **Isolasi Total:** Lapisan *database* (*Data Tier*) benar-benar disembunyikan di jaringan paling dalam, sehingga tidak ada celah untuk bisa diakses langsung dari internet luar.
- **Filter Serangan siber:** Di sinilah *Application Tier* beraksi sebagai garda depan. Lapisan ini berfungsi sebagai benteng yang menyaring dan menepis segala bentuk serangan siber berbahaya (seperti *SQL Injection*) sebelum sempat menyentuh atau merusak data penting perusahaan.

Kesimpulannya: Arsitektur *Three-Tier* adalah kunci utama agar sistem *enterprise* tetap lincah, tangguh, dan tidak boros biaya saat harus menangani lonjakan beban kerja yang ekstrem. Di saat yang sama, arsitektur ini memastikan aset digital paling berharga milik perusahaan tetap aman di balik sistem keamanan yang berlapis.

Soal 5:

Bayangkan sebuah perusahaan ingin mengubah logika perhitungan diskon pajak secara mendadak karena regulasi pemerintah baru. Dalam arsitektur *Three-Tier*, pada lapis (tier) manakah perubahan kode tersebut dilakukan? Mengapa pemisahan antara *Application Tier* dan *Data Tier* memudahkan proses pemeliharaan ini dibandingkan sistem tradisional yang menyatukan logika di dalam database?

Jawaban:

Ketika Regulasi Pajak Berubah Mendadak: Kenapa Memisahkan "*Application Tier*" dan "*Database*" Itu Penyelamat?

Bayangkan jika pemerintah tiba-tiba merilis aturan baru terkait diskon pajak secara mendadak. Jika sistem perusahaan sudah menggunakan arsitektur *Three-Tier*, tim *developer* hanya perlu mengutak-atik kode di **Application Tier** (Lapisan Aplikasi). Kenapa? Karena lapisan inilah yang bertindak sebagai "otak" utama yang menyimpan semua logika bisnis dan algoritma sistem.

Dibandingkan dengan sistem tradisional/jadul yang hobi menaruh logika bisnis langsung di dalam database (*stored procedures*), pemisahan ini membuat proses *maintenance* jadi jauh lebih manusiawi. Berikut adalah alasan logisnya:

1. Dampak Masalah Terisolasi (*Decoupling*)

Ketika ada perubahan rumus pajak di Lapisan Aplikasi, perubahan tersebut sama sekali tidak akan menyentuh atau merubah struktur tabel di dalam database. Pemisahan mutlak ini memperkecil risiko terjadinya data korup atau skenario terburuk di mana seluruh sistem mendadak lumpuh total akibat kesalahan koding.

2. Uji Coba Jauh Lebih Gampang & Aman (*Testing*)

Tim *developer* dan QA bisa menguji rumus pajak yang baru secara mandiri menggunakan data simulasi (*mock data*). Mereka tidak perlu repot-repot mereplikasi atau mengganggu database produksi (*production database*) yang ukurannya raksasa dan super sensitif hanya demi melihat apakah hitung-hitungannya sudah benar.

3. Update Sistem Tanpa Drama (*Zero-Downtime*)

Pembaruan kode pada server aplikasi bisa dilakukan secara bertahap (misalnya lewat metode *rolling update*). Hasilnya, sistem bisa diperbarui secara *live* tanpa harus mematikan layanan atau memasang pengumuman "*Situs Sedang Maintenance*". Operasional bisnis perusahaan pun tetap berjalan normal seolah tidak terjadi apa-apa.

4. Standarisasi Koding yang Lebih Modern

Logika bisnis di Lapisan Aplikasi ditulis menggunakan bahasa pemrograman umum yang populer (seperti Java, Python, atau Go). Bahasa-bahasa ini sangat mendukung ekosistem modern seperti pelacakan versi (*version control* menggunakan Git). Kondisi ini jauh lebih fleksibel dibanding sistem tradisional yang kaku karena harus tunduk pada bahasa database spesifik bawaan vendor tertentu.

Kesimpulannya: Pemisahan ini adalah kunci agar perusahaan bisa sangat lincah (*agile*) dalam mematuhi regulasi baru yang sering berubah mendadak. Proses pemeliharaan sistem pun berubah menjadi rutinitas yang cepat, aman, dan yang paling penting: tanpa perlu mengorbankan kelangsungan layanan bisnis.

Soal 6 :

Sistem ERP tradisional umumnya bersifat Monolithic (satu aplikasi besar). Namun, tren masa kini mulai beralih ke arsitektur Microservices (layanan kecil independen).

Diskusikan keuntungan arsitektur Microservices ditinjau dari aspek Availability dan Deployment! Apa tantangan terbesarnya dalam konteks integrasi data enterprise?

Jawaban:

Berikut adalah versi yang sudah diolah agar bahasanya mengalir lebih natural, dinamis, dan terasa seperti ditulis langsung oleh seorang *Tech Lead* atau *Enterprise Architect* berpengalaman:

Migrasi ERP ke Microservices: Sisi Indah dan Sisi "Pusing" yang Harus Diketahui

Beralih dari sistem ERP tradisional ke arsitektur *Microservices* itu ibarat mengubah satu kapal tanker raksasa menjadi armada sekoci kecil yang bergerak mandiri. Sistem tidak lagi menjadi satu aplikasi besar yang "gemuk", melainkan dipecah menjadi layanan-layanan kecil yang independen (seperti modul keuangan, gudang, atau penjualan yang berdiri sendiri) dan saling mengobrol via API.

Transisi ini membawa angin segar bagi efisiensi perusahaan, tetapi jujur saja, tantangan teknis di belakang layarnya juga tidak main-main.

Sisi Manisnya: Sistem Lebih Tangguh dan Gesit

1. Sistem Gak Gampang Tumbang (*High Availability*)

Arsitektur ini menerapkan prinsip isolasi kegagalan (*Fault Isolation*). Jadi, kalau modul laporan pajak mendadak *error* atau kena *bug*, layanan kritis lainnya seperti kasir (POS) atau manajemen stok bakal tetap menyala normal. Istilahnya, satu kamar bocor tidak akan membuat seluruh isi rumah banjir.

2. Bisa *Update* Kodingan Kapan Saja (*Agile Deployment*)

Tiap modul bisa diperbarui, dites, dan dirilis secara mandiri tanpa perlu repot-repot merilis ulang seluruh sistem ERP perusahaan. Biar tidak ada drama sistem *error* setelah *update*, tim IT bisa menggunakan teknik rilis bertahap seperti *Canary Deployment* (mencoba ke sebagian kecil pengguna dulu).

Sisi "Pusingnya": Mengelola Data Jadi Jauh Lebih Rumit

1. Kehilangan Konsistensi Data Instan (Selamat Tinggal ACID)

Karena setiap layanan wajib memegang *database*-nya sendiri (*Database-per-Service*), transaksi yang melibatkan lintas departemen jadi super ribet. Biar data antar-divisi tidak selisih atau tumpang tindih, tim *engineer* harus menerapkan pola arsitektur tingkat lanjut seperti *Saga Pattern*.

2. Bikin Laporan Eksekutif Jadi Lebih Menantang

Jajaran manajemen puncak tidak bisa lagi tinggal mengetik satu *query* SQL sederhana untuk menarik seluruh data perusahaan. Karena datanya terpecah di berbagai *database* yang berbeda, tim IT harus membangun pipa data (*Data Pipeline*) yang cukup rumit untuk menyatukan semuanya ke dalam sebuah *Data Warehouse*.

3. Masalah Latensi Jaringan

Berbeda dengan sistem tradisional yang komunikasinya instan karena berjalan di dalam satu memori komputer, *microservices* mengandalkan jaringan antar-API. Risiko jeda waktu (*delay*) dan potensi koneksi putus di tengah jalan otomatis menjadi tantangan harian yang harus diantisipasi.

Kesimpulannya: *Microservices* memberikan kecepatan inovasi dan ketahanan sistem yang luar biasa bagi skala *Enterprise*. Namun, ada harga yang harus dibayar: kerumitan tingkat tinggi

dalam mengelola dan menyinkronkan data perusahaan. Pastikan tim IT Anda memang sudah siap mental (dan *skill*) sebelum memutuskan untuk hijrah.

Soal 7 :

PT. Startup Logistik adalah perusahaan baru yang berkembang sangat cepat namun memiliki modal terbatas. Mereka butuh implementasi ERP dalam 1 bulan untuk menangani lonjakan pengiriman. Berdasarkan arsitektur teknis, model deployment mana yang Anda sarankan (Public Cloud atau On-Premise)? Berikan minimal 3 alasan strategis terkait biaya, waktu, dan pemeliharaan!

Jawaban:

Solusi Tepat untuk Lonjakan Pengiriman: Mengapa Public Cloud (SaaS) adalah Kunci

Menghadapi lonjakan pengiriman dengan modal cekak dan waktu yang cuma 1 bulan adalah situasi yang super kritis. Dalam kondisi ini, memaksakan diri menggunakan infrastruktur **On-Premise** sama saja dengan bunuh diri digital—risiko proyek gagal total sangat besar.

Pilihan paling realistis dan strategis jatuh pada **Public Cloud (SaaS)**. Berikut adalah 3 alasan kuat mengapa model ini adalah penyelamat bisnis kita saat ini:

1. Menjaga Napas Arus Kas (Geser Capex ke Opex)

Kita tidak punya kemewahan modal untuk dibuang di awal (*Capital Expenditure*) hanya demi beli *hardware*, server fisik, atau sewa ruang *data center*. Dengan Public Cloud, skemanya berubah menjadi *Operational Expenditure*—kita cukup bayar biaya langganan bulanan sesuai kapasitas yang benar-benar dipakai. Hasilnya? Dana segar perusahaan bisa dialokasikan untuk kebutuhan operasional lain yang lebih mendesak.

2. Kejar Tayang dalam Hitungan Jam (Kecepatan Deployment)

Tenggat waktu 1 bulan itu sangat sempit. Kalau pakai On-Premise, waktu kita akan habis hanya untuk urusan rakit kabel, instal OS, dan konfigurasi server dari nol. Di Public Cloud, infrastrukturnya sudah matang dan siap pakai dalam hitungan jam. Tim IT bisa langsung tancap gas melakukan migrasi data logistik dan melatih staf operasional tanpa buang-buang waktu.

3. Lepas Beban Teknis, Fokus pada Bisnis (Delegated IT)

Kita tidak perlu pusing memikirkan pemeliharaan perangkat keras, *update* keamanan, *backup* data harian, atau panik kalau sistem mendadak *down*. Semua jaminan kelancaran (*uptime* hingga 99,9%) sudah menjadi urusan dan tanggung jawab penuh vendor *cloud*. Tim IT kita yang jumlahnya terbatas bisa fokus 100% mengawal kelancaran bisnis logistik di lapangan.

Kesimpulan Praktis: Public Cloud memberikan tiga hal yang paling dibutuhkan PT. Startup Logistik saat ini: **kecepatan, efisiensi biaya, dan kepraktisan**. Ini adalah jalur cepat terbaik untuk mengamankan operasional bisnis tanpa harus menguras kantong di awal.

Soal 8:

Sebuah RS mengalami masalah: Pasien sudah bayar di kasir dan pulang, namun 2 jam kemudian bagian Farmasi baru menginput data obat yang diambil pasien tersebut, sehingga tagihan tidak tertagih. Menggunakan kacamata Real-time Integration, jelaskan di mana letak kegagalannya dan berikan usulan prosedur (Farmasi - Billing - Kasir) agar kerugian ini tidak terulang!

Jawaban:

Memutus Kebocoran Pendapatan RS: Transformasi dari Sistem Manual ke Real-Time Integration

Akar masalah dari kasus ini sebenarnya sederhana namun fatal: **data bergerak lebih lambat daripada pergerakan fisik pasien**. Karena rumah sakit masih mengandalkan pencatatan manual yang diinput secara berkala (*delayed/batch processing*), ditambah tidak adanya sistem penguncian tagihan otomatis antar-departemen, terjadilah *miss*. Kasir telanjur menutup transaksi pasien tanpa tahu bahwa di bagian Farmasi masih ada tagihan obat yang belum terinput.

Solusi mutlak untuk menyumbat kebocoran ini adalah dengan menerapkan prosedur berbasis **Real-time Integration**. Jadi, semua data bergerak instan mengikuti langkah pasien dengan alur baru seperti ini:

Alur Kerja Baru Berbasis Integrasi Instan

- **Langkah 1: Input Resep Digital (Dokter → E-Prescription)** Begitu dokter menginput resep secara digital, detik itu juga data langsung meluncur ke bagian Farmasi. Di saat yang sama, *Modul Billing* otomatis mengunci perkiraan biaya obat tersebut ke dalam draf tagihan pasien. Tidak ada lagi drama resep kertas terselip.
- **Langkah 2: Validasi & Penguncian Biaya (Farmasi → Penyiapan)** Petugas Farmasi menyiapkan obat dan memindai *barcode*-nya. Sistem *Billing* akan langsung memperbarui nilai tagihan menjadi final, sekaligus otomatis **mengunci status pasien** di sistem agar kasir tahu ada transaksi yang wajib dilunasi sebelum pasien pulang.

- **Langkah 3: Pembayaran Terpusat (Kasir → Pembayaran)** Pasien melakukan pembayaran total (biaya dokter + obat) di kasir. Begitu transaksi dinyatakan lunas, sistem secara *real-time* langsung melempar notifikasi berstatus "**Lunas/Paid**" ke layar komputer di bagian Farmasi.
- **Langkah 4: Penyerahan Obat & Potong Stok (Farmasi → Penyerahan)** Petugas Farmasi mengecek status lunas di layarnya, lalu menyerahkan obat kepada pasien. Bersamaan dengan itu, jumlah stok obat di gudang langsung berkurang otomatis secara akurat.

Kesimpulan Strategis: Dengan mengubah *golden rule* menjadi "**Bayar Dulu Baru Ambil Obat**" yang didukung integrasi sistem yang instan, rumah sakit tidak hanya berhasil menghentikan kebocoran pendapatan secara permanen, tetapi juga menjaga akurasi stok gudang dan menghindari risiko salah tagih ke pasien.

Soal 9 :

Analisis krisis yang terjadi di PT Nusantara kemudian berikan solusi konkret agar krisis yang terjadi tersebut tidak terulang kembali .

Profil Perusahaan

Nama Perusahaan: PT Nusantara Megastore (Perusahaan Retail Besar).

Sistem yang Digunakan: Aplikasi Core E-Commerce berbasis 3-Tier Architecture (Presentation, Application, Data Tier).

Kronologi Krisis (The Nightmare Event)

Pada pukul 00.00 di hari promosi terbesar tahunan "Mega Flash Sale 12.12", trafik platform melonjak 800% dalam waktu 3 menit.

1. Menit ke-1 (Presentation Tier Kolaps): Server web/thin client tidak mampu menangani antrean koneksi HTTP (pembatasan connection pool). Pengguna mulai melihat eror 502 Bad Gateway.
2. Menit ke-5 (Application Tier Bottleneck): Logika bisnis untuk pemrosesan pembayaran (payment gateway checkout) terkunci. Satu fungsi yang lambat menahan seluruh thread server

aplikasi, membuat fitur lain seperti pencarian produk ikut mati total.

3. Menit ke-10 (Database Tier Deadlock): Ribuan request yang menggantung mencoba menulis data ke satu database pusat secara bersamaan. Terjadi database deadlock pada tabel inventaris stok barang. CPU Server Database menyentuh 100%.

4. Dampak Finansial: Sistem mati total (downtime) selama 4 jam. Perusahaan kehilangan potensi pendapatan senilai Rp 12 Miliar dan mendapat sentimen negatif massal di media sosial.

Analisis Kegagalan Arsitektur (Titik Lemah 3-Tier Monolitik)

□ Skalabilitas Vertikal yang Kaku (No Elasticity): Arsitektur 3-Tier lama tidak bisa menambah server secara otomatis (auto-scaling) saat trafik melonjak ekstrem dalam hitungan menit.

□ Efek Domino (Single Point of Failure): Karena seluruh kode aplikasi menyatu (monolitik), kegagalan pada fungsi checkout pembayaran meruntuhkan seluruh sistem, termasuk fungsi katalog yang sebenarnya ringan.

□ Keterbatasan Database Tunggal: Database relasional konvensional tidak mampu menangani operasi Read/Write secara simultan dalam volume masif tanpa adanya pemisahan beban kerja (database sharding).

Jawaban:

Belajar dari Tragedi Flash Sale 12.12: Mengurai Akar Masalah dan Solusi Skalabilitas Sistem

Krisis kelumpuhan sistem selama 4 jam saat ajang Flash Sale 12.12 kemarin adalah alarm keras bagi perusahaan. Akar masalahnya jelas: **arsitektur 3-Tier Monolitik lama kita sudah terlalu kaku**. Ketika trafik tiba-tiba melonjak hingga 800%, terjadilah efek domino yang fatal di seluruh lapisan sistem:

- **Lapisan Presentasi** langsung tumbang karena kehabisan batas koneksi (*error 502 Bad Gateway*).
- **Lapisan Aplikasi** kehabisan *thread* kerja akibat fungsi pembayaran yang macet total.
- **Lapisan Data** mengalami *Deadlock* (gagal mengunci data stok secara bersamaan) hingga membuat beban CPU menyentuh angka 100%.

Dampaknya tidak main-main. Perusahaan harus menelan kerugian materiil sebesar Rp12 Miliar dan menghadapi hantaman sentimen negatif massal dari konsumen. Agar mimpi buruk ini tidak terulang, berikut adalah rekomendasi solusi konkret yang wajib segera kita eksekusi:

1. Solusi Arsitektur Teknologi (*System Upgrade*)

- **Lapisan Presentasi (*Cloud Auto-Scaling & CDN*):** Kita harus migrasi ke *Public Cloud* dan mengaktifkan fitur *Auto-Scaling Group*. Dengan begitu, jumlah server akan bertambah otomatis dalam hitungan menit saat trafik melonjak. Selain itu, pasang CDN (seperti Cloudflare) di gardu paling depan untuk menyaring dan meringankan beban *request* sebelum masuk ke server utama.
- **Lapisan Aplikasi (*Microservices & Circuit Breaker*):** Aplikasi monolitik yang raksasa itu harus dipecah menjadi layanan-layanan mandiri yang terpisah (Modul Katalog, Keranjang, dan Pembayaran harus berdiri sendiri). Kita juga perlu memasang pola *Circuit Breaker*. Tujuannya, jika modul pembayaran melambat atau *down*, modul katalog dan pencarian barang di aplikasi tetap bisa berjalan normal tanpa ikut mati total.
- **Lapisan Data (*Redis Caching & Message Queue*):** Manfaatkan Redis (*In-Memory Database*) untuk menangani proses pembacaan sisa stok agar tidak langsung menghantam database utama. Untuk proses penulisan data transaksi, gunakan *Message Queue* (seperti Apache Kafka atau RabbitMQ) agar antrean data masuk secara teratur dan rapi. Langkah ini akan mengeliminasi risiko *Deadlock* pada database.

2. Solusi Manajemen & Operasional (*People & Process*)

- *Load & Stress Testing* Berkala: Wajib hukumnya melakukan uji beban ekstrem dengan mensimulasikan trafik hingga 1000%. Ini harus dilakukan minimal sebulan sebelum *event* besar dimulai untuk mendeteksi titik jenuh sistem lebih awal.
- *Virtual Waiting Room*: Kita perlu menerapkan halaman antrean digital otomatis. Jika jumlah pengunjung sudah melewati ambang batas maksimal kemampuan server, sistem akan menahan sebagian user di ruang tunggu virtual demi menjaga situs utama tetap stabil.
- **Pembentukan *War Room*:** Menyiapkan ruang kendali khusus (*War Room*) yang diisi oleh tim IT lintas fungsi saat *event* berlangsung. Tujuannya agar kita bisa memantau metrik performa secara *real-time* dan mengambil tindakan instan jika ada anomali.

Kesimpulan Strategis: PT Nusantara Megastore sudah saatnya bertransformasi dari sistem monolitik terpusat ke arsitektur terdistribusi berbasis *cloud*. Langkah modernisasi ini akan mengisolasi setiap potensi kegagalan, mengotomatisasi skalabilitas server, dan melindungi database utama dari hantaman trafik massal secara bersamaan.